Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3582768.3582793

RESEARCH-ARTICLE

Vietnamese Text Summarization Based on Elementary Discourse Units

KHANG NHUT LAM, Can Tho University, Can Tho, Vietnam

TAI NGOC NGUYEN, Can Tho University, Can Tho, Vietnam

JUGAL KUMAR KALITA, University of Colorado Boulder, Boulder, CO, United States

Open Access Support provided by:

Can Tho University

University of Colorado Boulder

PDF Download
3582768.3582793.pdf
22 December 2025
Total Citations: 1
Total Downloads: 56

Published: 16 December 2022

Citation in BibTeX format

NLPIR 2022: 2022 6th International Conference on Natural Language Processing and Information Retrieval
December 16 - 18, 2022
Bangkok, Thailand

NLPIR '22: Proceedings of the 2022 6th International Conference on Natural Language Processing and Information Retrieval (December 2022)
https://doi.org/10.1145/3582768.3582793
ISBN: 9781450397629

Vietnamese Text Summarization Based on Elementary Discourse Units

Khang Nhut Lam
Can Tho University
Can Tho, Vietnam
lnkhang@ctu.edu.vn

Tai Ngoc Nguyen
Can Tho University
Can Tho, Vietnam
nguyenngoctai2007@gmail.com

Jugal Kalita
University of Colorado
Colorado Springs, USA
jkalita@uccs.edu

ABSTRACT

This paper presents text summarization models based on elementary discourse units (EDUs) to construct extractive and abstractive summarization for Vietnamese documents. First, we introduce algorithms using the POS information for constructing EDUs in Vietnamese. Then, the EDUs created are fed into an extractive summarization model using a pointer network and an abstractive summarization model using a pointer generator model. A reinforcement learning method is used to improve the quality of the models. We perform experiments on the CTUNLPSUM dataset, including 1,053,702 Vietnamese documents extracted from online magazines. The extractive summarization models based on EDUs outperform other extractive summarization models based on words or sentences. The ROUGE-1, ROUGE-2, and ROUGE-L of the best extractive and abstractive summarization models are 0.567, 0.241, 0.461; and 0.530, 0.213, 0.394, respectively.

CCS CONCEPTS

• **Information systems → Summarization**.

KEYWORDS

Extractive summarization, abstractive summarization, elementary discourse units, EDU segmentation, reinforcement learning, Vietnamese

ACM Reference Format:
Khang Nhut Lam, Tai Ngoc Nguyen, and Jugal Kalita. 2022. Vietnamese Text Summarization Based on Elementary Discourse Units. In *2022 6th International Conference on Natural Language Processing and Information Retrieval (NLPIR 2022), December 16–18, 2022, Bangkok, Thailand*. ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3582768.3582793

1 INTRODUCTION

Text summarization is the process of shortening an input document into a meaningful and seamless summary that contains the main ideas of the original document. Summaries are automatically constructed using extractive or abstractive text summarization approaches.

In general, the extractive summarization models compute the importance scores of sentences, phrases, or words in a document and concatenate them to create a brief version of the document. A common approach for extractive summarization is a graph-based method. In a graph-based method, sentences in a document are represented as nodes and the connections between nodes are weighted based on similarities. Erkan and Radev [5] introduced LexRank to generate extractive summaries by computing degree centrality of sentences in a sentence-level graph using intra-sentence cosine similarity. Another extractive summarization graph-based method [20] ranks sentences using the HITS [8] algorithm, ensures non-redundancy, computes local coherence using outdegree in a project graph, and optimizes these values. EdgeSumm [4] combines several extractive summarization models based on the graph, statistical, semantic, and centrality methods for constructing extractive summaries, and generates summaries using LSA and sentence clustering. Dong et al. [2] proposed an unsupervised model for extractive summarization of long scientific document, called HipoRank, by augmenting a graph-based ranking model using positional and hierarchical information grounded. HETERSUMGRAPH [27] combines the graph-based method with a neural network to construct extractive summaries. The graph initializer encodes nodes and edges, then the heterogeneous graph updates these nodes through a graph attention network [26], and the sentence selector predicts extractive sentences. With the explosion of neural networks in natural language processing, several extractive models based on neural networks have been introduced. The MATCHSUM framework [31] for extractive summarization matches the original documents and summary candidates into a semantic space. It obtains the similarity between documents and summaries using the Siamese-BERT architecture based on BERTSUM [14]. The position bias through the process of selecting sentences for extractive summarization is overcome by combining a conditional generation model with a set of sub-aspect functions [17] such as importance and diversity [7].

The extractive summarization models are simpler and faster than the abstractive summarization models. However, extractive summaries are less flexible, coherent, fluent, and comprehensible than abstractive summaries. Most abstractive summarization models follow the encoder-decoder architecture based on neural networks. The drawbacks of the abstractive summarization models include summaries not reflecting the document contents faithfully, and lack of control on aspects of the generated summary. In addition, the abstractive summaries created by neural network models may contain repetitive, incoherent and inconsistent words. To overcome issues in abstractive summarization models, several approaches have been proposed. BERTSUMABS [14] modifies a bidirectional Transformer encoder with masked language modeling to predict summaries.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
NLPIR 2022, December 16–18, 2022, Bangkok, Thailand
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9762-9/22/12…$15.00
https://doi.org/10.1145/3582768.3582793

GSum [3] improves a Transformer-based model for abstractive summarization by investigating the guidance signals of highlighted sentences, keywords, relations, and retrieved summaries. SimCLS [15] is a two-stage abstractive summarization model, which generates summary candidates using BART [10], then scores and predicts candidates using RoBERTa [16] assisted by contrastive learning based on original documents. Zhao et al. [30] improve the model by discovering semantic information. Their model comprises an LSTM for encoding documents, a variational RNN decoder for creating summaries, and a variational neural inferer for calculating the neural distribution probabilities. To improve word prediction, an abstractive summarization model, called ProphetNet [22], incorporates the original Transformer and n-stream self-attention mechanism to predict N-gram at each time step. In addition, several studies use reinforcement learning to improve abstractive summarization. In reinforcement learning, at each time step, an agent receives a state, performs an action, then receives an award, and transitions to the next state [11]. The agent maximizes the expectation of cumulative rewards. Reinforcement learning has also been used to improve the summarization models [21, 29]. In addition, some studies improve abstractive summarization by incorporating an extractive model into the abstractive summarization model. For example, a sentence-level extractive summarization may be used to modulate word-level abstractive summarization [6], such that the abstractive summarization model rewrites sentences extracted by the extractive model with the help of reinforcement learning [1].

Most existing summarization approaches are based on sentence-level or word-level, which might be difficult to generate summaries. Therefore, we present summarization approaches based on meaningful elements, each of which is smaller than a sentence but larger than a word, the so-called elementary discourse units (EDU) [28]. To the best of our knowledge, we have not found any EDU dataset in Vietnamese and any study for text summarization based on EDUs in Vietnamese. In summary, our contributions are three fold:

- We propose approaches to construct EDUs from Vietnamese documents.
- Several text summarization based on EDUs are performed to construct summaries from Vietnamese news articles.
- The extractive summarization models based on EDUs outperform other existing extractive summarization models based on words or sentences in Vietnamese.

2 EDU SEGMENTATION

A document is segmented into non-overlapping EDUs [28]. These EDUs can help the summarization system select the main ideas and remove unnecessary components. This section proposes approaches for creating EDUs in Vietnamese based on the Part-Of-Speech (POS) of Vietnamese words obtained from the Python Vietnamese toolkit[1].

By observing the available English EDUs[2] [28] and analyzing Vietnamese documents, the first temporary EDU segmentation approach based on the POSes of "C" (Conjunction) or "E" (Preposition) is presented in the Algorithm 1. Given an input sentence *sent*, a POS tagger toolkit is used to segment the *sent* into words and label them with corresponding POSes (line 1). A word w is a 2-tuple *<word,pos>*.

[1]https://pypi.org/project/pyvi/
[2]https://github.com/PKU-TANGENT/NeuralEDUSeg

The *EDUarray* initially has one element of empty string (line 2). For each word w_i in the sentence, if its $w_i.pos$ is "C" or "E", a new element containing that $w_i.word$ is appended to the *EDUarray* (line 3-5); otherwise, the last element of the current *EDUarray* is updated by concatenating the current value with a blank space " " and the $w_i.word$ (line 6-7). We remove the first element of the *EDUarray* if it is an empty string "" (line 8-9). In addition, if the first element of the *EDUarray* starts with a blank space " ", we remove this blank space (line 10-11).

Algorithm 1 EDU segmentation approach based on the POSes of "C" or "E"

Input: a sentence *sent*
Output: an array of EDUs $EDUarray$
1: POS tagging (*sent*)
2: EDUarray=["]
3: **for all** words w_i in *sent* **do**
4: **if** $w_i.pos$ = "C" || $w_i.pos$ = "E" **then**
5: EDUarray.append ($w_i.word$)
6: **else**
7: EDUarray [-1] ← EDUarray [-1] + " "+ $w_i.word$
8: **if** EDUarray[0]= "" **then**
9: remove $EDUarray[0]$
10: **if** EDUarray[0] starts with " " **then**
11: EDUarray[0].remove(" ")
12: **return** EDUarray

The second EDU segmentation approach based on the POSes of "C" (Conjunction) and the comma symbol "," along with "V" (Verb), "E" (Preposition), "N" (Common noun), or "R" (Adverb) is presented in the Algorithm 2. The second approach also goes through each word w_i in the *sent* (line 1-3). If w_i is a comma "," or the $w_i.pos$ is a "C" (line 4), and the next word w_{i+1} has a POS of "V" or "E" (line 5), we append a new element $w_i.word$ to the *EDUarray* (line 6). Similarly, $w_i.word$ is append to the *EDUarray* if w_i is a comma "," or the $w_i.pos$ is a "C"; the word w_{i+1} has a POS of "N"; and the word w_{i+2} has a POS of "V", "E" or "R" (line 7-9). Otherwise, we concatenate $w_i.word$ to the current value of last element of the *EDUarray* (line 10-11). The steps to remove unnecessary elements in the *EDUarray* are similar to steps in the Algorithm 1 (line 12-15).

After performing experiments on EDU segmentation approaches, we notice that the Algorithm 1 segments sentences by words or phrases; whereas the Algorithm 2 segments sentences by clauses. Besides, Mann and Thompson [19] defined EDUs as clauses [12]. We also perform small experiments to create summaries using EDUs generated by using the two algorithms proposed. The experiments show that the EDUs created using the Algorithm 2 help achieve better results with meaningful summaries. Therefore, we construct EDUs from the Vietnamese documents using the Algorithm 2.

3 EDU SELECTION

After segmenting EDUs from Vietnamese documents, the next step is to select EDUs for extractive summarization. Each sentence in the documents is assigned an order, the so-called *SENTord*. Each

Algorithm 2 EDU segmentation based on the POSes of "CE", "CV", "CN", ",NV", ",NE", and ",NR"

Input: a sentence *sent*
Output: an array of EDUs *EDUarray*
POS tagging (*sent*)
EDUarray=["]
for all words w_i in sent **do**
  if $w_i.pos$ = "C" || w_i = "," **then**
    if $w_{i+1}.pos$ = "V" || $w_{i+1}.pos$ = "E" **then**
      EDUarray.append ($w_i.word$)
    else if $w_{i+1}.pos$ = "N" **then**
      if $w_{i+2}.pos$ = ("V" || "E" || "R") **then**
        EDUarray.append ($w_i.word$)
  else
    EDUarray [-1] ← EDUarray [-1] + " "+ $w_i.word$
if EDUarray[0]= "" **then**
  remove $EDUarray[0]$
if EDUarray[0] starts with " " **then**
  EDUarray[0].remove(" ")
return EDUarray

EDU extracted in the previous section is unique and has a unique index, the so-called *EDUindex*. The Depth-First Search (DFS) algorithm [24] and ROUGE [13] metric are used to select EDUs for the extractive summarization model. We modify the method for selecting EDUs provides by Li et al. [12] as follows.

Step 1: Create an array $Indices_1$ consisting of the indices of EDUs with a depth of 2 and EDU indices in each element are sorted in ascending order. In other words, the $Indices_1$ includes all possible singleton sets $\bigcup \{[a, b] \mid a < b\}$, where a and b are indices of EDUs. For example, an *EDUarray* of [0,1,2,3] will have an array $Indices_1$ of [[0], [1], [2], [3], [0, 1], [0,2], [0,3], [1,2], [1,3], [2,3]].

Step 2: For each sentence $SentRef_i$ in the reference summary:

- Calculate the ROUGE score of each element in the $Indices_1$, by computing overlapping unigram between the element and $SentRef_i$, and select 5 elements with the highest ROUGE scores, the so-called *EDUcand*s. The idea of this step is to find the best EDUs. For example, the best 5 *EDUcand*s found are [0], [1], [3], [0,3], [1,3].
- Create a new array $Temp$ containing unique elements from the list of $EDUcand$s found in the previous step, and sort them in ascending order. In particular, the array $Temp$ is [0,1,3].
- Generate an array $Indices_2$ consisting of indices of elements in the $Temp$ with the depth of 5. The goal of this step is to increase the number of EDUs selected without increasing the depth step by step. In our example, the $Indices_2$ is [[0], [1], [3], [0, 1], [0,3], [1,3], [0,1,3]].
- Compute the ROUGE scores of elements in $Indices_2$ and select the element with the highest ROUGE score, the so-called *EDUbest*. For example, the *EDUbest* is [0,1,3].

Step 3: All elements in each $EDUbest$s extracted in the Step 2 are considered as a group and labeled as *EDUextracted*.

4 SUMMARIZATION MODELS BASED ON EDU

We follow the EDUSum model for abstractive summarization proposed by Li et al. [12]. The EDUSum model consists of an EDU Selection model for extractive summarization and an EDU Fusion model for abstractive summarization. The EDU Selection is followed a hierarchical encoder-decoder framework and based on the Pointer Network. In particular, the EDU Selection encoder is a word-level CNN and an EDU-level Bi-LSTM; and the EDU Selection decoder is a Bi-LSTM with an attention of Luong [18]. The EDU Selection decoder computes the distribution probabilities of EDUs s_i^t at time step t, and groups related EDUs Sel_t, each of which is separated by a label *truncate*, using the following formula.

$$s_i^t = \begin{cases} v_p^T tanh\left(W_p\left[c_t; h_i\right]\right) & i \text{ not in } Sel_t \\ -\infty & \text{otherwise} \end{cases} \quad (1)$$

where i is the EDU index, v_p and W_p are the training parameters, c_t is the context.

The EDU Fusion model based on the Pointer Generator [23] is used to construct an abstractive summary sentence by copying words from a group of EDUs extracted from the EDU Selection. Both EDU Selection and EDU Fusion models are pre-trained by optimizing for maximum probabilities. After obtaining 2 models, a reinforcement learning method based on the Advantage Actor-Critic algorithm [1] is used to improve the selection of EDUs of the EDU Selection, which is an agent. At each decoder step, the agent performs an action to select an EDU based on their selection probabilities. The agent receives an award for a group of EDU extracted based on the ROUGE-L score computed by the overlapping words between the abstractive summary generated by the EDU Fusion and the reference summary.

5 EXPERIMENTS

5.1 Data Preprocessing

We perform experiments on 1,053,702 documents from the CTUNLP-SUM dataset [9], which consists of articles on 20 topics collected from Vietnamese common online newspapers, such as vnexpress.net, tuoitre.vn, dantri.com.vn, danviet.vn, laodong.vn, vietnamnet.vn. The reference summary of an article includes the article title and the first following paragraph as presented in Figure 1.

Noisy words such as html tags and special symbols in documents are removed. The ViPosTagger of the pyvi[3] library is used to segment words. Words in documents are standardized to the utf-8 encoding. Each document is saved in a file such that each line is a sentence and the reference summary is separated by the token *@highlight*.

First, each sentence is segmented into EDUs. Next, the DFS is used to create training data consisting of the EDU indices of an abstractive summary sentence, the so-called *EDUextracted* and ROUGE score of elements of *EDUextracted*. We eliminate documents comprising less than 3 sentences or greater than 100 EDUs because too short or too long documents might cause inference. In addition, we notice that the documents with too small ROUGE scores might contain less extractable information; whereas documents with too high ROUGE scores might have summary sentences that are pretty similar to the sentences in the document content. Therefore, we

[3]https://pypi.org/project/pyvi/

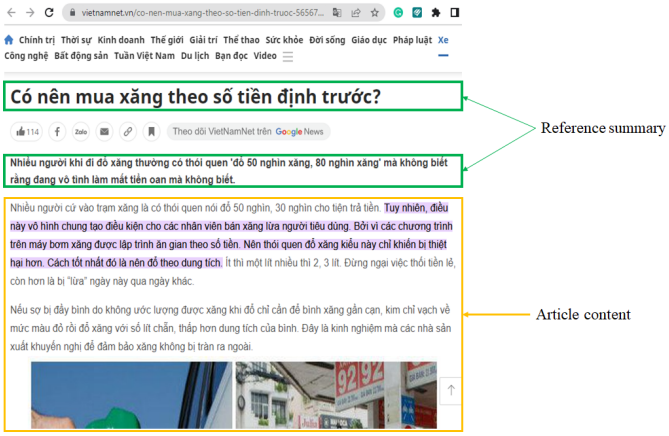

Figure 1: An online article from vietnamnet.vn.

remove documents with ROUGE scores less than α or greater than β. In our experiments, α=0.3 and β=0.85 help remove meaningless documents. After the pre-processing step, we have 628,773 documents with 5,360,824 sentences. We use 100,000 documents for testing; 100,000 documents for validation; and the remaining documents for training. The average number of EDU extracted from a sentence is 2.07 and the average number of tokens in an EDU is 12.69. The longest EDU and abstractive summary constructed have 100 tokens and 200 tokens, respectively.

5.2 Experiment Setup

EDU Selection and EDU Fusion models are trained on a machine with CPU core i7-6700, GPU (GeForce GTX 1070 8GB). The EDUSum model is trained on Google Colaboratory with Intel Xeon CPU configuration, GPU (Tesla K80).

The EDU Selection and EDU Fusion models are trained and checkpoints updated after 3,000 steps, training batch size of 16, and Adam optimizer. The dimensions of hidden layer and skip-gram embedding are 256 and 128, respectively. The other parameter of models are as follows: learning rate: 0.001, decay ratio: 0.5, gradient clipping: 2.0, beam size: 5, and maximum number of sentences: 60. Both models will stop the training process when 5 consecutive checkpoints can no longer be learned.

The EDUSum model is trained and checkpoints updated after 1,000 steps. We use the same parameters suggested by Li et al. [12]: a learning rate of 0.0001, a discount factor of 0.95, and Adam optimizer.

5.3 Experimental Results

We evaluate the models on 2 test sets:

- CTUNLPSUM [9] test set consists of 2,000 Vietnamese documents.
- ViMs [25] test set includes 1,900 Vietnamese documents.

First, we evaluate the extractive summarization models using the EDU Selection model, and the EDU Selection model with reinforcement learning method. The ROUGE scores of these models are presented in Table 1 and Table 2.

Table 1: Extractive summarization using EDU Selection

Dataset	ROUGE	ROUGE-1	ROUGE-2	ROUGE-L
CTUNLPSUM	Recall	0.517	0.211	0.404
	Precision	0.636	0.258	0.499
	F1-score	0.549	0.223	0.430
ViMs	Recall	0.575	0.250	0.424
	Precision	0.581	0.244	0.427
	F1-score	0.542	0.231	0.398

Table 2: Extractive summarization using EDU Selection with reinforcement learning

Dataset	ROUGE	ROUGE-1	ROUGE-2	ROUGE-L
CTUNLPSUM	Recall	0.716	0.308	0.584
	Precision	0.493	0.208	0.400
	F1-score	0.567	0.241	0.461
ViMs	Recall	0.758	0.359	0.602
	Precision	0.417	0.188	0.326
	F1-score	0.515	0.236	0.405

The ROUGE scores of EDU Fusion and EDUSum for abstractive summarization are presented in Table 3 and Table 4, respectively.

Table 3: Abstractive summarization using EDU Fusion

Dataset	ROUGE	ROUGE-1	ROUGE-2	ROUGE-L
CTUNLPSUM	Recall	0.448	0.158	0.307
	Precision	0.604	0.220	0.424
	F1-score	0.497	0.178	0.344
ViMs	Recall	0.497	0.183	0.313
	Precision	0.572	0.215	0.363
	F1-score	0.515	0.191	0.325

Table 4: Abstractive summarization using EDUSum

Dataset	ROUGE	ROUGE-1	ROUGE-2	ROUGE-L
CTUNLPSUM	Recall	0.504	0.202	0.373
	Precision	0.615	0.251	0.464
	F1-score	0.530	0.213	0.394
ViMs	Recall	0.562	0.233	0.388
	Precision	0.552	0.228	0.385
	F1-score	0.525	0.215	0.362

The experimental results show that reinforcement learning helps improve the EDU Selection model for extractive summarization. Moreover, extractive summarization models based on EDUs outperforms other extractive summarization models based on sentences or words [9] such as RNN, GRU-RNN, LSTM, BiLSTM, and BERT on the

same CTUNLPSUM dataset. Figure 2 presents F1-scores of several extractive summarization models performed on the CTUNLPSUM dataset.

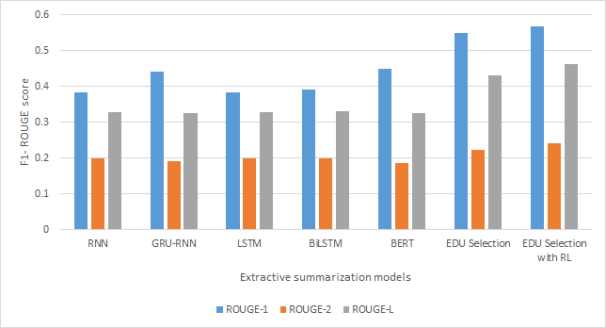

Figure 2: ROUGE scores of extractive summarization models

The EDUSum abstractive summarization model performs better than the EDU Fusion abstractive summarization model. In addition, these two models are better than the abstractive summarization models at word-level using neural networks with attention such as RNN with attention and BERT abstractive in Vietnamese. However, the EDUSum and EDU Fusion models are not as good as other abstractive summarization models with copy generator and bottom-up approach [9], which has the best ROUGE-1, 2, and L scores of 0.598, 0.260, and 0.455, respectively. The EDU Selection works well for extractive summarization; while the EDU Fusion needs to improve. For future work, we will study a method to handle the summary prediction of EDU Fusion, or discover an abstractive summarization based on EDUs using the bottom-up approach to take advantages of both bottom-up approach and the EDUs.

6 CONCLUSION

We propose approaches for constructing EDUs for text summarization models in Vietnamese. The extractive summarization model based on EDUs outperforms all existing extractive summarization models based on words or sentences. The reinforcement learning method helps improve the results of the extractive summarization model, thereby helping improve the results of the abstractive summarization model. The summary sentences generated by the EDUSum model seems smoother and more meaningful than the EDU Fusion model. We will ask volunteers to evaluate EDUs and summaries created. We are developing more efficient methods for segmenting EDUs, such as applying feature selection from POS tags and parse trees, and/or using neural discourse segmentor. We are also studying abstractive summarization models for exploiting the information-intensive of EDUs.

REFERENCES

[1] Yen-Chun Chen and Mohit Bansal. 2018. Fast abstractive summarization with reinforce-selected sentence rewriting. *arXiv preprint arXiv:1805.11080* (2018).
[2] Yue Dong, Andrei Romascanu, and Jackie CK Cheung. 2020. Hiporank: incorporating hierarchical and positional information into graph-based unsupervised long document extractive summarization. *arXiv preprint arXiv:2005.00513* (2020).
[3] Zi-Yi Dou, Pengfei Liu, Hiroaki Hayashi, Zhengbao Jiang, and Graham Neubig. 2020. Gsum: A general framework for guided neural abstractive summarization. *arXiv preprint arXiv:2010.08014* (2020).
[4] Wafaa S El-Kassas, Cherif R Salama, Ahmed A Rafea, and Hoda K Mohamed. 2020. EdgeSumm: graph-based framework for automatic text summarization. *Information Processing & Management* 57, 6 (2020), 102264.
[5] Günes Erkan and Dragomir R Radev. 2004. Lexrank: graph-based lexical centrality as salience in text summarization. *Journal of artificial intelligence research* 22 (2004), 457–479.
[6] Wan-Ting Hsu, Chieh-Kai Lin, Ming-Ying Lee, Kerui Min, Jing Tang, and Min Sun. 2018. A unified model for extractive and abstractive summarization using inconsistency loss. *arXiv preprint arXiv:1805.06266* (2018).
[7] Taehee Jung, Dongyeop Kang, Lucas Mentch, and Eduard Hovy. 2019. Earlier isn't always better: sub-aspect analysis on corpus and system biases in summarization. *arXiv preprint arXiv:1908.11723* (2019).
[8] Jon M Kleinberg. 1999. Authoritative sources in a hyperlinked environment. *Journal of the ACM (JACM)* 46, 5 (1999), 604–632.
[9] Khang Nhut Lam, Tuong Thanh Do, Nguyet-Hue Thi Pham, and Jugal Kalita. 2022. Vietnamese text summarization based on neural network models. In *International Conference on Artificial Intelligence and Big Data in Digital Era*. Springer, 85–96.
[10] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2019. Bart: denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461* (2019).
[11] Yuxi Li. 2017. Deep reinforcement learning: An overview. *arXiv preprint arXiv:1701.07274* (2017).
[12] Zhenwen Li, Wenhao Wu, and Sujian Li. 2020. Composing elementary discourse units in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 6191–6196.
[13] Chin-Yew Lin. 2004. Rouge: a package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.
[14] Yang Liu and Mirella Lapata. 2019. Text summarization with pretrained encoders. *arXiv preprint arXiv:1908.08345* (2019).
[15] Yixin Liu and Pengfei Liu. 2021. SimCLS: a simple framework for contrastive learning of abstractive summarization. *arXiv preprint arXiv:2106.01890* (2021).
[16] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: a robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
[17] Zhengyuan Liu, Ke Shi, and Nancy F Chen. 2020. Conditional neural generation using sub-aspect functions for extractive news summarization. *arXiv preprint arXiv:2004.13983* (2020).
[18] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. 2015. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025* (2015).
[19] William C Mann and Sandra A Thompson. 1988. Rhetorical structure theory: Toward a functional theory of text organization. *Text-interdisciplinary Journal for the Study of Discourse* 8, 3 (1988), 243–281.
[20] Daraksha Parveen and Michael Strube. 2015. Integrating importance, non-redundancy and coherence in graph-based extractive summarization. In *Twenty-Fourth International Joint Conference on Artificial Intelligence*.
[21] Romain Paulus, Caiming Xiong, and Richard Socher. 2017. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304* (2017).
[22] Weizhen Qi, Yu Yan, Yeyun Gong, Dayiheng Liu, Nan Duan, Jiusheng Chen, Ruofei Zhang, and Ming Zhou. 2020. Prophetnet: predicting future n-gram for sequence-to-sequence pre-training. *arXiv preprint arXiv:2001.04063* (2020).
[23] Abigail See, Peter J Liu, and Christopher D Manning. 2017. Get to the point: summarization with pointer-generator networks. *arXiv preprint arXiv:1704.04368* (2017).
[24] Robert Tarjan. 1972. Depth-first search and linear graph algorithms. *SIAM journal on computing* 1, 2 (1972), 146–160.
[25] Nhi-Thao Tran, Minh-Quoc Nghiem, Nhung TH Nguyen, Ngan Luu-Thuy Nguyen, Nam Van Chi, and Dien Dinh. 2020. ViMs: a high-quality Vietnamese dataset for abstractive multi-document summarization. *Language Resources and Evaluation* 54, 4 (2020), 893–920.
[26] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
[27] Danqing Wang, Pengfei Liu, Yining Zheng, Xipeng Qiu, and Xuanjing Huang. 2020. Heterogeneous graph neural networks for extractive document summarization. *arXiv preprint arXiv:2004.12393* (2020).
[28] Yizhong Wang, Sujian Li, and Jingfeng Yang. 2018. Toward fast and accurate neural discourse segmentation. *arXiv preprint arXiv:1808.09147* (2018).
[29] Xingxing Zhang and Mirella Lapata. 2017. Sentence simplification with deep reinforcement learning. *arXiv preprint arXiv:1703.10931* (2017).
[30] Huan Zhao, Jie Cao, Mingquan Xu, and Jian Lu. 2020. Variational neural decoder for abstractive text summarization. *Computer Science and Information Systems* 17, 2 (2020), 537–552.
[31] Ming Zhong, Pengfei Liu, Yiran Chen, Danqing Wang, Xipeng Qiu, and Xuanjing Huang. 2020. Extractive summarization as text matching. *arXiv preprint arXiv:2004.08795* (2020).